



ScaWL: Scaling k -WL (Weisfeiler-Lehman) Algorithms in Memory and Performance on Shared and Distributed-Memory Systems

COBY SOSS, Washington State University, Pullman, United States and Schweitzer Engineering Laboratories Inc, Pullman, United States

ARAVIND SUKUMARAN RAJAM, Meta, Menlo Park, United States

JANET LAYNE, Boise State University, Boise, United States

EDOARDO SERRA, Boise State University, Boise, United States

MAHANTESH HALAPPANAVAR, Data Sciences and Machine Intelligence, Pacific Northwest National Laboratory, Richland, United States

ASSEFAW H. GEBREMEDHIN, Washington State University, Pullman, United States

The k -dimensional Weisfeiler-Lehman (k -WL) algorithm—developed as an efficient heuristic for testing if two graphs are isomorphic—is a fundamental kernel for node embedding in the emerging field of graph neural networks. Unfortunately, the k -WL algorithm has exponential storage requirements, limiting the size of graphs that can be handled. This work presents a novel k -WL scheme with a storage requirement orders of magnitude lower while maintaining the same accuracy as the original k -WL algorithm. Due to the reduced storage requirement, our scheme allows for processing much bigger graphs than previously possible on a single compute node. For even bigger graphs, we provide the first distributed-memory implementation. Our k -WL scheme also has significantly reduced communication volume and offers high scalability. Our experimental results demonstrate that our approach is significantly faster and has superior scalability compared to five other implementations employing state-of-the-art techniques.

CCS Concepts: • **Computing methodologies** → **Parallel algorithms; Distributed algorithms; Concurrent algorithms; Machine learning algorithms;**

Additional Key Words and Phrases: Optimization, k -WL algorithm, parallel

This research was supported by the PATH program at Schweitzer Engineering Laboratories and by the National Science Foundation through Award Number 2126449. At Pacific Northwest National Laboratory, this work is also supported by the U.S. Department of Energy through the Exascale Computing Project (17-SC-20-SC) (ExaGraph).

Authors' Contact Information: Coby Soss, Washington State University, Pullman, Washington, United States and Schweitzer Engineering Laboratories Inc, Pullman, Washington, United States; e-mail: coby.soss@wsu.edu; Aravind Sukumaran Rajam, Meta, Menlo Park, California, United States; e-mail: aravindsr@meta.com; Janet Layne, Boise State University, Boise, Idaho, United States; e-mail: janetlayne@boisestate.edu; Edoardo Serra, Boise State University, Boise, Idaho, United States; e-mail: edoardoserra@boisestate.edu; Mahantesh Halappanavar, Data Sciences and Machine Intelligence, Pacific Northwest National Laboratory, Richland, Washington, United States; e-mail: hala@pnnl.gov; Assefaw H. Gebremedhin, Washington State University, Pullman, Washington, United States; e-mail: assefaw.gebremedhin@wsu.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 1544-3973/2025/03-ART45

<https://doi.org/10.1145/3715124>

ACM Reference Format:

Coby Soss, Aravind Sukumaran Rajam, Janet Layne, Edoardo Serra, Mahantesh Halappanavar, and Assefaw H. Gebremedhin. 2025. ScaWL: Scaling k-WL (Weisfeiler-Lehman) Algorithms in Memory and Performance on Shared and Distributed-Memory Systems. *ACM Trans. Arch. Code Optim.* 22, 1, Article 45 (March 2025), 25 pages. <https://doi.org/10.1145/3715124>

1 Introduction

Graphs are important data structures used to represent objects (nodes) and their relationships to one another (edges). Machine learning on graphs is enabled via graph representation learning, which automatically assigns numerical vectors to nodes, edges, subgraphs, or even entire graphs. The goal of graph representation learning is to replace hand-engineered features with learned representations that encode information about the relationships between entities in the graph [10]. Graph isomorphism testing determines whether two graphs with different node identities are structurally the same. The utility of these methods in generating representations that capture graph structure is intuitive; and we single out the **Weisfeiler-Lehman (WL)** isomorphism test as an efficient approximation of the graph isomorphism test [27].

The WL algorithm is foundational to many graph representation learning approaches, including standard **Graph Neural Networks (GNNs)**, many of which employ message passing [21]. GNNs currently dominate the field of graph representation learning, and recent studies have shown that **message-passing neural networks (MPNN)** have a ceiling for expressivity equivalent to the 1-WL algorithm [21, 28]. Intuitively, the 1-WL algorithm iteratively determines the equivalence-class label—commonly referred to as color—of a vertex by hashing the multiset of the current labels of its neighbors. Given an initial label $l_v^{(0)}$ for vertex $v \in V$, the label at step i is given by:

$$l_v^{(i)} = \text{HASH}(\langle l_u^{(i-1)} \forall u \in \text{adj}(v) \rangle), \quad (1)$$

where the initial label $l_v^{(0)} = d(v)$, can simply be the degree $d \equiv |\text{adj}(v)|$, the number of nodes adjacent to v . The notation $\langle \rangle$ represents a multiset, and the $\text{HASH}(\langle \rangle)$ function maps a unique multiset to a unique new label. After n rounds of re-labeling, the label $l_v^{(n)}$ of a given node captures the summary of the n -hop neighborhood of v .

The WL kernel has been studied extensively and is demonstrated to capture many theoretical properties of learning [21]. It also has several applications in which the underlying problem of graph isomorphism needs to be solved [25]. Graph isomorphism is used in a wide range of areas, including social network analysis, computer security, compilers, and cheminformatics [9].

Building on the Weisfeiler-Lehman kernel, a generic *neural message passing* iteration in a GNN model can be expressed as:

$$\mathbf{h}_u^{(k+1)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_u^{(k)}, \text{AGGREGATE}^{(k)}(\langle \mathbf{h}_v^{(k)} \forall v \in \text{adj}(u) \rangle)\right) \quad (2)$$

where UPDATE and AGGREGATE functions are arbitrary differentiable functions, and $\mathbf{h}_v^{(k)}$ is the aggregated message from all the neighbors of u . Thus, using the message passing model, GNN can be defined as:

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)} \mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)} \sum_{v \in \text{adj}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\right), \quad (3)$$

where $\mathbf{W}_{\text{self}}^{(k)}, \mathbf{W}_{\text{neigh}}^{(k)} \in \mathbb{R}^{d^k \times d^{(k-1)}}$ are trainable parameters, σ is a nonlinear activation function, and $\mathbf{b}^{(k)} \in \mathbb{R}^{d^k}$ is the weighted bias. Different types of neighborhood normalizers, aggregators,

and attention heads are used for different types of GNNs. Consequently, different architectures have different complexity and purpose [21].

Both Xu et al. and Morris et al. demonstrate that Equation (2) corresponds to the hashing function of the 1-WL in message-passing-based GNNs [20, 28]. As a consequence, any graphs (or nodes, subgraphs) considered “possibly isomorphic” by the 1-WL will share identical representations resulting from Equation (2) (disregarding node features).

Higher-order k -WL based approaches, where $k > 1$, increase the expressivity of representations but also entail a *large increase in the computational and memory requirements*. As described in Section 4, existing implementations of the k -WL algorithm suffer from massive network communication costs, data movement costs, and data footprint requirements and are, therefore, not scalable. In this work, *we develop an efficient k -WL algorithm that addresses these challenges*.

Our proposed ScaWL approach is guided by insights into the k -WL algorithm and delivers a data footprint and communication-optimized single-node k -WL implementation (Section 5.2) and a distributed k -WL implementation (Section 5.3). Our results demonstrate good internode and intranode scaling and outperform existing implementations (Section 6).

Contributions: We make the following key contributions:

- Enable efficient processing of large graphs, thereby achieving high performance.
- Reduce storage requirements by an order of magnitude.
- Develop a communication-avoiding algorithm for distributed processing.
- Present empirical evaluations that demonstrate robust intranode and internode scaling, and superior performance of ScaWL compared to five state-of-the-art WL-based techniques.
- Present a formal proof showing the equivalence between ScaWL and the classic k -WL algorithm in terms of expressivity (Section 7).

To the best of our knowledge, ours is the first scalable, distributed implementation of the k -WL algorithm that also minimizes memory footprint. We, therefore, believe that this work will enable the use of k -WL in GNNs for a wide variety of applications that critically depend on accurate node embeddings. The source code for ScaWL is located at <https://github.com/CobySoss/ScaWL>.

2 Related Works

The influence of the WL-hierarchy of algorithms is found in many contexts. Serra et al. [24] developed SparseStruct, a WL-inspired algorithm to generate numerical node representations for vertices in large graphs. Campedelli et al. [4] adapted the algorithm to analyze terrorist network graphs, enabling graph-to-terrorist-organization prediction. The works of Joaristi and Serra [12, 13] present SIR-GN and SILA, respectively. Both algorithms, similar to 1-WL, iteratively learn information about nodes based on their neighborhood structure. Shervashidze et al. [26] use the 1-WL algorithm as a basis for a graph kernel that utilizes the similarity determination of 1-WL to classify graphs.

GNNs also exemplify concepts similar to the 1-WL algorithm. Like 1-WL, GNNs iteratively aggregate node neighborhoods, integrating information into the node’s features at each iteration. Sato [21] concludes that the 1-WL algorithm is more powerful at distinguishing two graphs than **message-passing GNNs (MPNNs)** because the aggregation and update functions may be non-injective. Xu et al. [28] addressed this limitation by making these functions injective and developed the **Graph Isomorphism Network (GIN)**, which shows expressive power comparable to 1-WL and formally demonstrated that the 1-WL algorithm is the upper bound for the expressive power of MPNNs. Several works [5, 16, 19, 20] propose novel variations of GNNs relying on higher-order WL methods. For example, Morris et al. [19] developed higher-order WL algorithms, the δ - k family of WL-inspired kernels and neural architectures. The local versions of these algorithms only

consider a subset of the nodes' neighborhoods to overcome the prohibitive time and space complexity of the algorithm, and all were shown to have strictly more expressive power than k -WL.

Bodnar et al. [3] introduce a **Simplicial Weisfeiler-Lehman (SWL)** coloring procedure to distinguish non-isomorphic simplicial complexes. They show that this procedure can distinguish strongly regular graphs. Based on SWL, they developed their **Message Passing Simplicial Network (MPSN)** method. Both SWL and MPSN are strictly more powerful than the 1-WL algorithm and no less powerful than the 3-WL algorithm. Sato et al. [22] show that using random features can enhance expressivity of GNNs. This approach, however, may undermine an invariance property where two entities that share identical neighborhood structures will be assigned identical representations. Zhang et al. [30] conduct a thorough evaluation of the expressivity of GNNs under different definitions of expressivity, including but not limited to the graph isomorphism perspective.

Yan et al. [29] propose a framework called CycleNet, which uses the encoding of cycle information in graphs to help distinguish graphs that contain such cycles. CycleNet is strictly more powerful than 2-WL, and it can distinguish graphs that 3-WL and 4-WL cannot. Zhang et al. [31] introduce an expressivity metric called homomorphism expressivity, which provides a fine-grained and quantitative method to evaluate GNN expressiveness. Homomorphism expressivity distinguishes between the expressivity of GNNs in ways that can be missed by the WL hierarchy, such as subgraph counting. Morris et al. [17] provide an extensive discussion about supervised machine learning, elucidating the deep theoretical links between the 1-WL algorithm and GNNs, positioning 1-WL as a foundational starting point for evaluating GNN performance.

An analysis of the rate of convergence of the 2-WL algorithm is provided by Lichter et al. [15], showing that the algorithm converges in $O(n \log n)$ iterations. Fürer [8] found the lower bound of the algorithm to be $\Omega(n)$ for degree-3 graphs.

In sharp contrast to existing work, we focus on a scalable and distributed implementation of the k -WL algorithm while minimizing memory requirements for larger values of k in order to enable wider adoption of the k -WL algorithm in applications. In addition, our insights support future works inspired by the WL-hierarchy.

3 Background

The k -WL algorithm is a family of increasingly powerful approximations to the graph isomorphism problem. A high-level description of the k -WL algorithm, adopted from Sato [21], is presented in Algorithm 1. The goal of the k -WL algorithm is to compare k -tuple color multisets to distinguish between two graphs. The process can be summarized as k -tuple color initialization followed by one or more iterations of neighborhood discovery, new color generation from neighborhood information, and k -tuple color multiset comparisons between the two graphs.

First, the algorithm initializes the k -tuples of the two graphs based on the connectivity of the vertices within each k -tuple (Lines 1 and 2). Each iteration of the k -WL algorithm computes the colors of all k -tuples (Lines 7 and 9), and the k -tuple color multisets of the two graphs are compared (Line 4). The color of a k -tuple depends on the structural information contained in its k fibers. The term fiber refers to the neighborhood function $\mathcal{N}_{G,i}$, defined shortly (Lines 6 and 8). If the k -tuple color multisets of both graphs G and H do not match, the graphs are not isomorphic, and the algorithm returns false (Line 5). The algorithm proceeds to the next iteration if they match, and returns true, or possible isomorphic, if the k -tuple color multisets for the graphs match for two consecutive iterations without growing in size.

The algorithm produces a multiset of colors for each $c_{v,i}^{(l)}$ and $d_{u,i}^{(l)}$ using the definition of the neighborhood $\mathcal{N}$ of each k -tuple u or v . The general definition of neighborhood is $\mathcal{N}_{G,i}((v_1, v_2, \dots, v_k)) = \{(v_1, \dots, v_{i-1}, w, v_{i+1}, \dots, v_k) | w \in V\}$. In other words, the i th neighborhood of a

ALGORITHM 1: k -WL Algorithm [21]

Result: Possibly isomorphic/not isomorphic (true/false)
Params: Graph $G = (V, E)$, Graph $H = (U, F)$

```

1  $c_v^{(0)} \leftarrow \text{HASH}\langle G[v] : \forall v \in V^k \rangle$ 
2  $d_u^{(0)} \leftarrow \text{HASH}\langle H[u] : \forall u \in U^k \rangle$ 
3 for  $l = 1, 2, \dots$  to convergence do
4   if  $\{\{c_v^{(l-1)} | v \in V^k\}\} \neq \{\{d_u^{(l-1)} | u \in U^k\}\}$  then
5     return false (not isomorphic)
6    $c_{v,i}^{(l)} \leftarrow \{\{c_w^{(l-1)} | w \in \mathcal{N}_{G,i}(v)\}\} (\forall v \in V^k, i \in [k])$ 
7    $c_v^l \leftarrow \text{HASH}\langle c_v^{l-1}, c_{v,1}^l, c_{v,2}^l, \dots, c_{v,k}^l : \forall v \in V \rangle$ 
8    $d_{u,i}^{(l)} \leftarrow \{\{d_w^{(l-1)} | w \in \mathcal{N}_{H,i}(u)\}\} (\forall u \in U^k, i \in [k])$ 
9    $d_u^l \leftarrow \text{HASH}\langle d_u^{l-1}, d_{u,1}^l, d_{u,2}^l, \dots, d_{u,k}^l : \forall u \in U \rangle$ 
10 return true (possibly isomorphic)
```

k -tuple is the color multiset derived by replacing the i th index of the tuple with every vertex in the graph [21]. The notation V^k (similarly U^k) denotes the Cartesian product of the set V operated k times (similarly for U).

This paper uses a shorthand for the formulation of the hash keys in lines 7 and 9. Notice that a tuple's neighborhood generation in the 2-WL case is simply the column colors (obtained by varying v_i) concatenated with the row colors (obtained by varying v_j). To finish creating the key, we simply prepend the previous color of 2-tuple (v_i, v_j) . Therefore, the shorthand in our examples for key creation becomes the concatenation of the previous iteration's color ($\langle i, j \rangle$), column colors ($\langle *, j \rangle$), and row colors ($\langle i, * \rangle$). The terms i and j are the matrix indices representing the 2-tuples in our 2-WL implementation. In Algorithm 1, the 2-WL previous color, column colors, and row colors for graphs G and H are $c_v^{l-1}, c_{v,1}^l, c_{v,2}^l$ and $d_u^{l-1}, d_{u,1}^l, d_{u,2}^l$, respectively. While this expression aids in explaining the new algorithm introduced in this paper, it is simply a rephrasing of the official 2-WL formulation. This shorthand formulation is identical for 3-WL but with the added dimension concatenated at the end of tuple keys for G and H , namely $c_{v,3}^l$ and $d_{u,3}^l$, respectively.

To illustrate the workings of the k -WL algorithm, we use the examples in Figures 1 and 2, which show the initial state and the state after the first iteration for the 2-WL case, maintaining a color for every pair of nodes. More generally, the k -WL algorithm maintains a color for every tuple of nodes with length k . Graph initialization can be done using different approaches. In this work, we initialize the color to 1 (connected) for all pairs of nodes with an edge between them and 0 (not connected) otherwise. Figure 2 shows the colors after the first iteration, where color 0 is represented by black, color 1 by blue, color 2 by red, color 3 by orange, and color 4 by brown. The coloring function should be injective per iteration, i.e., all the tuples with the same color and the same k fibers in the current iteration should receive the same color in the next iteration.

4 Challenges

The efficacy of the k -WL algorithm is limited by multiple factors. Foremost among these challenges is the substantial storage requirement, which curtails the algorithm's capacity to process massive graphs. Computational efficiency related to effective data volume constitutes another pivotal challenge. Load balancing, both intranode and internode work distribution, introduces a further challenge. Internode communication volume represents a critical challenge, potentially nullifying the advantages of distributed computing efforts.

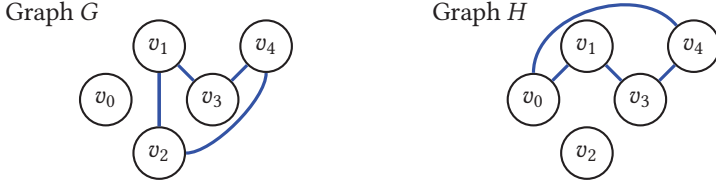


Fig. 1. Initial 2-tuple coloring of the 2-WL Algorithm, represented by colored edges (color 1) or their absence (color 0).

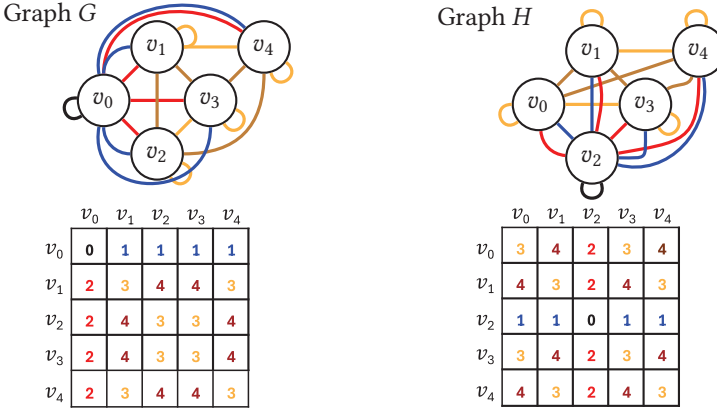


Fig. 2. 2-tuple coloring after one iteration of the 2-WL Algorithm. Top: Graphs G and H with colors represented by edges. Bottom: 2-WL color matrix for G and H .

Storage Requirement. Recall that the k -WL algorithm requires that the color of two k -tuples, which have the same color neighborhood, be the same and that the color of two k -tuples with a different color neighborhood be different. Each k -tuple requires $O(k|V|)$ space to store the neighborhood colors, resulting in a total space requirement of $O(k|V|^{k+1})$ for storing the colors of $|V|^k$ k -tuples. This exponential space requirement limits the size of graphs that can be processed. As described in Section 5, our algorithm reduces the storage cost to $O(k|V|^k)$, which is an order of magnitude improvement.

Data-movement and data access efficiency. Each iteration of the k -WL algorithm computes the colors for $|V|^k$ tuples, each of which requires reading $|V| * k$ data elements, resulting in $O(k|V|^{k+1})$ element-wise data accesses. The color is then computed using an injective hash function that only involves a few operations. Reading $O(k|V|^{k+1})$ elements and only performing a few operations on them results in very low **operational intensity (OI)**. Thus, the performance of k -WL is significantly limited by memory bandwidth on all modern architectures. Note that the OI is substantially lower than **Sparse Matrix-Dense Vector multiplication (SpMV)**, which is well known as a bandwidth-bound kernel. To achieve good memory access efficiency, a single thread should access contiguous data (for spatial reuse) on CPUs. Moreover, the algorithm accesses data elements on all k dimensions. Using a single data representation implies that only one among the k dimensions will be contiguous, and the rest will be non-contiguous, significantly impacting data access efficiency. For example, when $k = 2$, we need to access the elements along rows and columns, and only the elements along the rows are contiguously laid out in memory.

Load imbalance. Injective hashing is one key component of the k -WL algorithm that significantly affects performance. Unfortunately, the work associated with each hash is dependent on the current state of the hash table, especially the hash bin, and hence is inherently load imbalanced.

Communication volume. A naive distributed implementation of the k -WL algorithm requires broadcasting the hash values, which results in $O(k|V|^{k+1}p)$ communication volume, where p is the number of processors. This huge communication volume will limit distributed scaling. As described in Section 5, our algorithm avoids this communication and drastically reduces the overall communication cost down to $O(k|V|^{k-1}p)$.

5 ScaWL

We describe the proposed ScaWL algorithm in detail in this section. We first provide an overview in Section 5.1, and then discuss the single node implementation in Section 5.2 and the distributed node implementation in Section 5.3.

5.1 Overview

Figure 3 shows an overview of the single-node ScaWL framework. First, we load and partition the data across different threads. The partitioning is based on the hash value of each of the k dimensions, which helps to improve load balancing and reduce inter-thread communication (details in Section 5.2). Next, we construct the color of each k -tuple. Our novel approach computes the k -tuple color using the fiber colors along each of the k dimensions, which helps in reducing the data movement and storage requirement. We then check whether the k -tuple colors of the two graphs match; if they do not match, the graphs are not isomorphic. If the colors match and the number of new colors is greater than the color count of the previous iteration, we proceed to the next iteration; if the number of colors remains the same across two iterations, the graphs are possibly isomorphic and the algorithm terminates. **It is important to note that a significant proportion (99%) of the computational time is spent on computing colors.** Therefore, it is critical to improve it to achieve higher performance.

5.2 k -WL on a Single Node

We describe the ScaWL single-node algorithm for the case where $k = 2$ for simplicity of explanation. However, our approach is general and will work for any value of k .

In Algorithm 1, the most computationally expensive step is computing the colors of each vertex tuple ($\{ \langle i, j \rangle : 0 \leq i, j < |V| \}$ for 2-WL). The coloring function should be injective, which implies that tuples whose previous color, the color of column neighbors, and the color of row neighbors are the same should have the same color. Hence, only injective hashing functions (no collisions) can be used to assign colors. A naive way to achieve this is to construct a string using the k -WL algorithm's key generation formulation, the concatenation of the previous color, the sorted column colors, and the sorted row colors for 2-WL. Each such unique string represents a unique color. Figure 4 illustrates this process. For example, consider tuple $\langle 1, 2 \rangle$, whose color string is $7_0_0_0_2_3_4_7_0_0_1_1_1_2_7$. Once the color string is constructed, the injective hash table is searched for a match. If there is a hit, the corresponding entry is used as the color of the tuple; if not, a new color is inserted. In our example, the color string of tuple $\langle 1, 2 \rangle$ does not match existing values in the hash table; hence, a new color is assigned to it, and a corresponding entry will be added to the hash table. Note that tuple $\langle 3, 3 \rangle$ has the same color string as $\langle 1, 2 \rangle$, and hence will be assigned the same color.

The entire string needs to be kept during the construction of colors so that one can map each unique string to a unique integer. The length of each entry in the hash table is $2|V| + 1$ (hence,

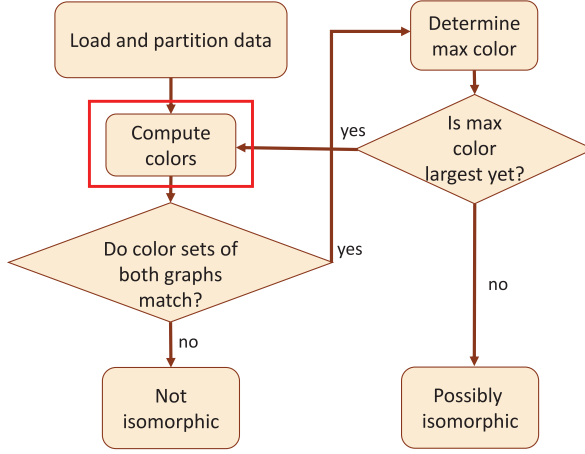


Fig. 3. Overview of ScaWL. “Compute colors” is the most computationally expensive step.

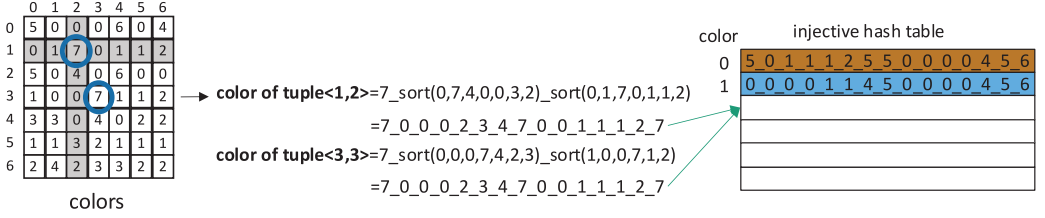


Fig. 4. Hashing and coloring scheme of the standard k -WL algorithm.

$O(|V|)$), and the total space requirement is $O(|V|^3)$ for 2-WL and $O(|V|^{k+1})$ for k -WL for general k . This space requirement limits the size of the graph that can be processed using this scheme. Our approach, ScaWL, reduces both storage and computational costs without compromising precision.

5.2.1 The ScaWL Coloring Algorithm. The ScaWL approach realizes that the color string for each k -tuple is the concatenation of the previous color, column color, and row color. The challenge is generating an injective mapping for each row and column to an integer. Computing row/column colors must avoid storing intermediate color strings. Once the mapping is computed, we can assign each tuple a unique color using an approach similar to the original algorithm where the color string can be represented with just three integer values (current color, column color, row color) as opposed to $2|V| + 1$ values, contributing to ScaWL’s reduced storage requirement.

Algorithm 2 shows how each row and column can be converted to unique integers efficiently without using a memory-intensive hash table. The ScaWL coloring algorithm achieves this by grouping rows (or columns) into bins with a hash function. This grouping ensures members outside the bin cannot match, reducing the number of comparisons needed for coloring; the rest of this section explains the various steps in Algorithm 2, which has four main parts: initializing the row and column colors, partitioning the rows and columns into bins, assigning threads to process those bins, and finally coloring the rows and columns within the bins. The initialization of row and column colors assumes all rows and all columns are unique and colors them accordingly.

Binning. Assigning a unique color to each row by comparing each row to others requires no additional temporary space. However, it does require significant computational resources, having a comparison complexity of $O(|V|^3)$ for 2-WL and $O(|V|^{k-1}|V|^{k-1}|V|) = O(|V|^{2k-1})$ for k -WL for

ALGORITHM 2: ScaWL coloring algorithm

Result: Color per row and column for 2-WL
Input: tuplesG1, tuplesG2, $|V|$
Output: rowColorG1[$|V|$], colColorG1[$|V|$], rowColorG2[$|V|$], colColorG2[$|V|$],
1 initializeRowAndColColors(*tuplesG1*, *tuplesG2*)
2 bins $\leftarrow$ initBins(*tuplesG1*, *tuplesG2*)
3 workSplit[] $\leftarrow$ partitionWork(*bins*)
4 colorRowsAndCols(*bins*, *workSplit*, *tuplesG1*, *tuplesG2*, $|V|$)

ALGORITHM 3: Group rows (columns) into bins

Result: Bin id per fiber
Input: tuplesG1, tuplesG2, $|V|$
Output: bins.row[], bins.col[]
1 **Function** initBins():
2 $\triangleright$ For each row
3 **parallel for** $r \leftarrow 0$ to $|V|$ **do**
4 $result1 \leftarrow 0$
5 $result2 \leftarrow 0$ $\triangleright$ We used two commutative hashes, but more can be used.
6 **for** $c \leftarrow 0$ to $|V|$ **do**
7 $result1 \leftarrow \text{commutative_hash_f1}(result1, \text{tuplesG1.color}(r, c))$
8 $result2 \leftarrow \text{commutative_hash_f2}(result2, \text{tuplesG1.color}(r, c))$
9 $hash \leftarrow (\text{bitMask1}(result1) \mid \text{bitMask2}(result2))$
10 bins.row[*hash*].append(*r*);
11 $\triangleright$ Similar for tuplesG2
12 $\triangleright$ Repeat above steps for columns

general k . To reduce this comparison computation, we employ the binning process described in Algorithm 3. It works by grouping only potentially matching rows (or columns) together, reducing the required comparisons. First, we group the rows based on the hash of all the colors of the 2-tuples in that row, or the row multiset. The hash function here may be **non-injective**. In other words, identical row multisets will hash to the same value, but distinct row multisets may also hash to the same value, placing them in the same group (bin). The functions *sum* and *XOR* meet the non-injective hashing requirement and are computationally inexpensive.

For example, consider rows 3 and 4 in Figure 5; though they have the same hash (14), these rows should be assigned different colors because the sorted colors of their 2-tuples differ. Rows 0 and 2 have identical hashes and sorted 2-tuple colors. These observations illustrate that *only* rows within the same group (bin) need to be compared. This filtering saves computations as well as data movement. We can aggregate the results of multiple non-injective hash functions, minimizing false positives and enhancing the quality of distinction. The ScaWL implementation combines *sum* and *XOR* functions, selecting alternate bits from each respectively to create the hash function.

Work Partitioning. Algorithm 4 describes the assignment of bins to the threads that will do the comparison work. To load-balance the work across the bins, the algorithm sorts the bins according to their workload and distributes the bins across threads in a round-robin fashion to ensure efficient processing. However, load imbalance can still occur if there are either a small number of large bins or many large bins that vary significantly in size. We discuss a solution to this problem in Section 5.2.2.

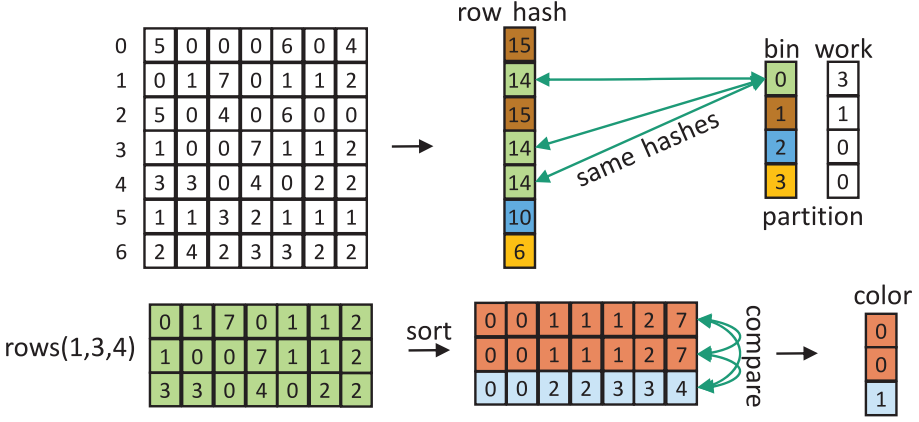


Fig. 5. ScaWL work partitioning and row coloring.

ALGORITHM 4: Partition work across different threads**Result:** Starting and ending bin IDs for each thread**Input:** bins[]**Output:** workSplit[]

```

1 Function partitionWork(bins):
2   ▷ Sort bins by work
3   for  $i \leftarrow 1$  to bins.row.size do
4      $n \leftarrow$  bins.row[i].size
5      $binWork \leftarrow n(n-1)/2$ 
6      $workPerBin[i] \leftarrow binWork$ 
7   sortByWork(bins, workPerBin)
8   ▷ Round-robin assign threads
9    $tid \leftarrow 0$ 
10  for  $i \leftarrow 1$  to bins.row.size do
11     $workSplit.row[tid] \leftarrow i$ 
12     $tid++$ 
13    if  $tid > numThreads$  then
14       $tid \leftarrow 0$ 
15  ▷ Similar for columns

```

Coloring of Rows and Columns. Algorithm 5 details how colors are assigned to rows (or columns) by performing the actual comparison work that has been set up by the binning and thread assignment algorithms. For example, rows 1 and 3 within the bin in Figure 5 can be considered similar if they have equal color multisets, such as $\langle 0, 1, 7, 0, 1, 1, 2 \rangle$ and $\langle 1, 0, 0, 7, 1, 1, 2 \rangle$. The column ID of each 2-tuple color is irrelevant to the hash functions described in Algorithm 3.

To make comparing rows easier, we sort the rows and then compare their 2-tuple colors left-to-right. Within a bin, we compare all the rows with each other. This requires $\binom{n}{2}$ row-wise comparisons, where n is the number of rows in the bin. If a source row is already colored, we skip it. If the target row is similar to the source row, the source row's color is assigned to the target row. Upon completion of Algorithm 5, each row is assigned a color such that every distinct multiset of 2-tuples in a row corresponds to a unique color assignment.

ALGORITHM 5: Color each row and column**Result:** All rows and columns for G and H will be assigned a color**Input:** bins[], workSplit[], tuplesG1, tuplesG2, |V|**Output:** Output is stored in tuplesG1 and tuplesG2

```

1 Function colorRowsAndCols(...):
2   parallel for  $tid \leftarrow 0$  to  $nThreads-1$  do
3      $startBin \leftarrow workSplit[tid].rowBin$ 
4      $endBin \leftarrow workSplit[tid+1].rowBin$ 
5     for  $binID \leftarrow startBin$  to  $endBin-1$  do
6       for  $r \in bin[binID].rows$  do
7         if  $r \in tuplesG1$  then
8            $sortRow(tuplesG1.row[r]);$  ▷ Sort colors in each row
9         else
10           $sortRow(tuplesG2.row[r]);$ 
11       for  $i \in bin[binID].rows$  do
12          $tuplesG_i \leftarrow (i \in tuplesG1) ? tuplesG1 : tuplesG2$ 
13         if  $tuplesG_i.rows.isProcessed[i]$  then
14            $continue$  ▷ Skip already colored rows
15         for  $j \in bin[binID].rows$  AND  $j > i$  do
16            $tuplesG_j \leftarrow (j \in tuplesG1) ? tuplesG1 : tuplesG2$ 
17            $match \leftarrow True$ 
18           ▷ If the color of every element in row  $i$  and  $j$  match, then assign row  $i$  and  $j$  the same color.
19           for  $col \in 0$  to  $|V|-1$  do
20             if  $tuplesG_i.elem.colorID[i][col] \neq tuplesG_j.elem.colorID[j][col]$  then
21                $match \leftarrow False$ 
22                $break$ 
23           if  $match$  then
24              $oldColor \leftarrow tuplesG_j.rows.colorID[j]$ 
25              $tuplesG_j.rows.colorID[j] \leftarrow tuplesG_i.rows.colorID[i]$ 
26              $tuplesG_j.rows.isProcessed[j] \leftarrow True$ 
27             ▷ If this is a large bin, leverage preprocessing.
28             if  $large\ bin$  then
29               for  $k \in bin[binID].rows$  AND  $k > j$  do
30                  $tuplesG_k \leftarrow (k \in tuplesG1) ? tuplesG1 : tuplesG2$ 
31                 if  $tuplesG_k.rows.colorID[k] == oldColor$  then
32                    $tuplesG_k.rows.colorID[k] \leftarrow curColor$ 
33                    $tuplesG_k.rows.isProcessed[k] \leftarrow True$ 
34              $tuplesG_i.rows.isProcessed[i] \leftarrow True$ 
35   ▷ Similar for columns

```

In summary, our ScaWL approach allows us to represent the k -WL hash key from lines 7 and 9 of Algorithm 1 using three values for 2-WL and $1 + k$ values for the general case k -WL, resulting in a storage cost of $O(|V|^k)$ – an order of magnitude lower than prior approaches. Computing the row/column colors also avoids multiple scans during element processing, reducing data movement.

Next, we explain the load-balancing steps that we take during different phases of the algorithm. The two main phases on a single node are column/row coloring and k -tuple coloring. We begin with a load-balancing strategy for column/row coloring.

5.2.2 Optimizing Load Balance in the Row/Column Coloring Phase. In this section we discuss our algorithm using rows but it applies equivalently to columns. As mentioned earlier, we distribute the processing of each row bin across different threads. However, a small number of large bins or many large bins which vary significantly in size can cause load imbalance. Our initial strategy assumes that all potentially identical rows fall into the same bin, making bin splitting difficult. To address this challenge, we employ a large bin processing algorithm that is broken into three phases: Bin splitting across multiple threads to reduce comparisons in each bin, coloring each smaller bin, and finally, merging all bins together again. The merging step is integrated into the row/column coloring algorithm (Algorithm 5: Line 28).

First, we partition large bins into smaller ones for independent processing. Within each small bin, the first row of each color is designated as the leader, reducing comparisons to just these leaders. After coloring each small bin, we merge the row/column colors to ensure consistent color assignment across the original large bin. For every leader row in each small bin, a comparison is made with leaders from other bins. If they match, one of the leaders, along with all rows represented by the leader, inherits the color of the other. For instance, if row r_1 in small bin bin_1 and r_2 in small bin bin_2 share the same row color, r_2 and the rows it represents are assigned the color of r_1 . This streamlined approach significantly reduces the total number of comparisons because of the efficiency of leader-to-leader comparisons. Once the rows (and columns) are colored, the next step is k -tuple coloring. The worst-scenario time complexity for row/column coloring happens in the case where every bin is a large bin of unique colors. This would prevent large bin processing from saving any work since the merge phase would find no matching colors to mark as finished coloring, and thus all rows/columns would require comparison. There will be $\binom{n}{2}|V|$ comparisons for processing each small bin, where t is the number of threads, n is the number of rows in the large bin, and $|V|$ is the number element comparisons per row. Since we have assumed, in the worst case, that all rows are unique (i.e., leaders), there will be an additional $\binom{n}{2}|V|$ comparisons during the merge phase. Therefore, the complexity per large bin is $O((\binom{n}{2} + \binom{n}{2})|V|) \approx O(n^2|V|)$.

5.2.3 Optimizing Load Balancing in the Tuple Coloring Stage. The aim of the load-balancing strategy during the tuple coloring stage is to divide the workload of the k -WL key hashing process as evenly as possible across compute cores while minimizing the number of hash table merges that are required. After the row/column coloring phase and before the tuple coloring stage, threads, each with their own hash table, take ownership of distinct rows using a hash of the row's color. This row partitioning ensures that no two 2-WL hash keys, which incorporate the row color, will be the same across threads. In other words, hash tables amongst different threads do not need merging if there is an ideally balanced load distribution. However, this initial distribution can be significantly skewed after the row/column coloring merge phase if many rows are assigned the same color (refer to Section 5.2.2), leading to substantial load imbalance.

To ensure a more balanced workload, we compute the standard deviation using the number of rows assigned to each thread. For threads with excessively high workload, we redistribute their work to one or more underutilized threads. The threads participating in a hash-table split are tracked. Each thread independently assigns color to each of its assigned tuples. For the threads participating in a split, the tuple colors may not be injective hash-consistent, as independent processing of two tuples with the same previous color, column color, and row color may result in different color assignments for each tuple. Therefore, we must merge each hash-table split to restore injective-hash consistency. This merging process resembles the merge in the row/column coloring phase. This load-balancing strategy ensures that a split occurs only under significant load imbalance, avoiding merge costs when the workload is reasonably balanced.

5.3 k -WL on Distributed Nodes

In this section, we explain our ScaWL distributed solution. To our knowledge, this is the first distributed implementation of the k -WL algorithm. First, we examine the limitations of a simplistic extension of the single-node solution for distributed systems. Then, we explain our innovative, communication-avoiding approach that significantly reduces the communication volume.

Limitations of simple extension of single-node solution. Consider an approach where rows are distributed across nodes, and we are on the n^{th} iteration. For computing row color on the $n + 1^{\text{th}}$ iteration, each node can independently sort its rows and compute the commutative hash function to determine each row's bin ID. Nodes exchange metadata on bin IDs and partition the row bins. The sorted rows and their bin IDs are sent to the bin owner for coloring, ensuring that each row is assigned an injective-hash-consistent color. *In the worst case, this leads to a $O(|V|^k)$ communication cost per iteration.*

Communication-avoiding algorithm. To enhance performance, we must minimize communication between nodes. We achieve this by developing a communication-avoiding algorithm [2, 7, 11] (note that such algorithms do not necessarily eliminate communication entirely).

We claim that once binning is done, no row in the bin will ever need to be compared with any row outside it. Our claim is based on the observation that the k -WL algorithm is refining tuple color in every iteration. Once two tuples or rows are assigned different colors, they remain distinct in all subsequent iterations. To better understand this claim, consider two tuples, $t_1 < i_1, j_1 >$ with row color c_1 and $t_2 < i_2, j_2 >$ with row color c_2 in the first iteration. If c_1 and c_2 are identical, they must belong to the same row bin. Our work partitioning approach ensures that the rows which belong to the same bin are assigned to the same processing node. The ScaWL coloring algorithm guarantees color consistency within the node. If c_1 is not equal to c_2 , then t_1 and t_2 can be in any processing nodes.

If t_1 and t_2 are in different processing nodes, the communicative row hash (Algorithm 3:Line 7) of i_1 and i_2 could match in a future iteration, mapping these rows to the same bin (Algorithm 3). However, they will receive distinct colors after processing (see Algorithm 5). Thus, placing them in the same bin is unnecessary. To understand why, we need to look at the 2-WL hash keys of t_1 and t_2 ; let color “*_ c_1 ” represent the hash key for t_1 , and “*_ c_2 ” represent the hash key for t_2 , where “*” represents any value (as a reminder, the 2-WL hash key is of the form “previous-Color_colColor_rowColor”). Since we have assumed that c_1 and c_2 are different in the current iteration, there is at least one element in rows i_1 and i_2 that won't match. This implies that in all subsequent iterations, these rows won't receive the same color. This observation allows us to avoid communication of rows in all iterations but the first. Figure 6 illustrates this approach. After the first iteration, only the row and column colors need to be exchanged. Hence, the communication cost per iteration (except the first iteration) is $O(k|V|^{k-1}p)$ as opposed to $O(|V|^k p)$ communication volume.

However, directly employing the above algorithm may yield suboptimal results due to internode load imbalance. In such cases, redistributing the workload can improve outcomes. We explain next the internode load-balancing heuristic that aims to minimize redistribution to mitigate communication overhead.

Internode load balancing. The internode balancing heuristic aims to strike a delicate balance between the time costs associated with data transfer and the computational time required to execute the algorithm. In essence, the objective is to guarantee a well-proportioned distribution of data

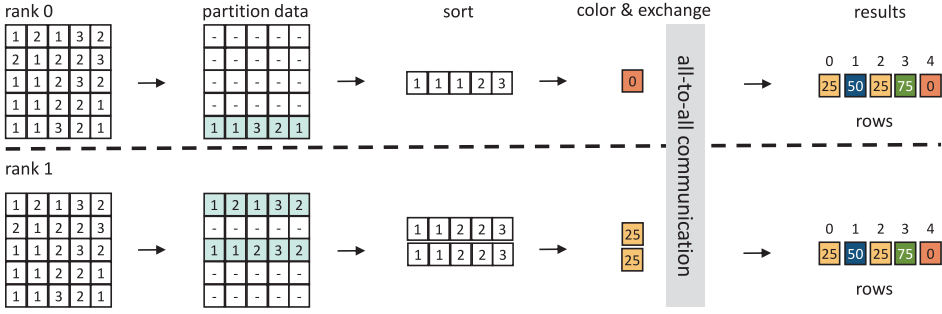


Fig. 6. ScaWL distributed scheme.

across nodes, preventing undue data transfers that could surpass the temporal costs of tolerating the existing imbalance and progressing forward.

Two metrics are utilized to determine the necessity of work redistribution. Note that in a perfectly load-balanced scenario, all nodes would have workloads close to the average. The first metric, **Mean Absolute Deviation (MAD)** is computed as the sum of the absolute differences between the workload per node and the average workload. MAD serves as a measure of how far individual workloads deviate from the average workload. The second, **minMaxDelta**, quantifies the disparity in workload between the node with the maximum workload and the one with the minimum workload ($\text{max_work} - \text{min_work}$). If either of these metrics exceeds a predefined threshold (we have set it at 35%) in any iteration, we initiate a workload redistribution.

The remaining steps of the load balancing algorithm closely mirror those used in Algorithm 5. Each node computes its total workload across all bins and then engages in all-to-all communication to ascertain the total workload for all nodes. The bins are subsequently sorted based on the amount of work, and we strive to redistribute the work so that each node maintains a workload close to the average. This involves identifying sender-receiver pairs and initiating point-to-point communication for data exchange.

6 Experimental Results

To evaluate the performance of the 2-WL and 3-WL algorithms comprehensively, we conducted a series of experiments on a cluster featuring an Intel Haswell architecture. Our investigation covered a range of graph sizes to showcase the scalability of these algorithms. This section presents performance results obtained from experiments on both single nodes and distributed nodes.

6.1 Experimental Setup

The experiments were conducted on an eight-node cluster, where each node had two Intel Xeon e5-2660 v3 processors running at 2.60 GHz and 256 GB of RAM. The code was compiled using GCC v7.3.0 and the experimental datasets are from the University of Florida Sparse Matrix Collection [6]. Table 1 provides details on the graphs.

We used MPI for internode parallelism and OpenMP for intranode parallelism. We present single-node scaling on 2, 4, 8, 16, and 20 cores. In distributed experiments, we showcase results using 2, 4, and 8 nodes, with each node running on 20 cores. The speeds achieved with one core and one node serve as our 1x reference speed for single-node and multi-node tests, respectively. If a multi-node reference-speed test run did not complete, we used the first successful test run as the 1x reference speed. To our knowledge, this work marks the first evaluation of the performance of the k-WL algorithm.

Table 1. Datasets used in the Evaluation

Graph	Vertices	Edges	MinD	MaxD
ct20stif	52,329	1,375,396	1	13
bcsstk32	44,609	1,029,655	0	134
pwt	36,519	181,313	1	15
onetonel	36,057	341,088	1	161
nd12k	36,000	7,128,473	1	480
lhr34	35,152	764,014	1	63
bcsstm35	30,237	18,211	0	6
wang3	26,064	177,168	4	7
crystk03	24,696	887,937	1	42
g7jac080sc	23,670	293,976	2	153
bcsstk36	23,052	583,096	1	116
pkstutk01	22,044	500,712	1	114
raefsky3	21,200	1,488,768	32	80
bayer04	20,545	159,082	1	43
olafu	16,146	515,651	1	68
poli_large	15,575	33,074	1	491

Graph	Vertices	Edges	MinD	MaxD
hangGlider_4	15,561	79,088	0	6918
bcsstk25	15,439	133,840	1	44
epb1	14,734	95,053	3	9
appu	14,000	1,853,104	12	294
coater2	9,540	207,308	0	63
Kaufhold	8,765	42,471	1	28
dw8192	8,192	41,746	3	8
G61	7,000	17,148	0	14
bcsstk27	1,224	56,126	1	33
1138_bus	1,138	4,054	1	17
b_dyn	1,089	4,264	1	11
bcsstk08	1,074	12,960	1	335
can_1072	1,072	6,758	1	32
msc01050	1,050	15,103	1	126
saylr3	1,000	2,375	1	4
cdde3	961	4,681	3	5

Graph	Vertices	Edges	MinD	MaxD
gr_30_30	900	7,744	1	5
orsirr_2	886	5,970	3	14
qh882	882	3,354	1	14
ex22	839	22,715	8	62
bp_1000	822	4,661	1	308
G12	800	1,600	0	4
fs_760_1	760	11,192	2	21
685_bus	685	3,249	1	13
fs_680_3	680	4,612	1	14
662_bus	662	2,474	1	9
494_bus	494	1,666	1	7
rsa480	480	34,134	4	119
celegansneural	297	4,690	0	134
Trefethen_20	20	89	6	6
LEAT5	14	30	2	5

The table shows the number of vertices, the number of edges, the minimum degree, and the maximum degree in each graph.

6.2 Comparison with State of the art

We utilized five state-of-the-art implementations of the k -WL algorithm for benchmarking: WL-Test [14], K-WL [23], **GraphTester (GT)** [1], GlobalWL [18], and SparseWL [19]. Table 2 shows the raw runtime and speedup results we obtained. Time or memory constraints limited our baseline dataset to smaller graphs. WL-Test encountered out-of-memory errors on all but one of the test graphs, whereas k -WL, GraphTester, GlobalWL, and SparseWL successfully completed most tests. The K-WL, GraphTester, GlobalWL, and SparseWL algorithms exhibited significantly slower performance, even when compared to our single-node, single-core implementation. GlobalWL was faster than the other benchmarks, partly due to programming language differences. However, ScaWL was faster on all but the very smallest graphs. Where GlobalWL outperforms ScaWL on small graphs, it is likely due to the overhead of MPI. Both a trivial (T) and a light (L) modification of GlobalWL were necessary, while only the trivial modification was performed on SparseWL.

The trivial modification involved altering the algorithms to run to convergence instead of a user-specified number of iterations. The light modification involved replacing the pairwise hashing function, which squares once per color in a k -tuple's k neighborhoods, with the hash table from ScaWL. This was done to prevent 64-bit overflow of the color when GlobalWL was hashing large graphs. For SparseWL, the pairwise hash use was more complex. Therefore, only trivial modifications were performed to avoid inadvertently altering the algorithm's behavior. In the trivial benchmark (GlobalWL-T and SparseWL-T), we allowed GlobalWL and SparseWL colors to overflow until convergence. In the light-modification benchmark (GlobalWL-L), we fixed the overflow issue. The light modification caused a slowdown because of the required storage and lookup of the hash key. The previous hash, being purely calculated, did not require storage or lookup operations.

6.3 Single Node Results

In this section, we present the scaling performance of 2-WL and 3-WL on a single node. For each input graph, we generate an isomorphic version using the following steps. If the graph is symmetric,

Table 2. Raw Run Times (Left of Double Lines) and Speedups Attained by ScaWL Single-node-single-core Versus State-of-the-art Implementations (Right of Double Lines)

Graph	WL-Test (s)	K-WL (s)	ScaWL (s)	vs K-WL
662_bus	OOM	15,859.37	7.23	2,193.6
494_bus	OOM	6,377.89	3.57	1,786.5
rbas480	OOM	2,202.19	3.03	726.8
celegansneural	OOM	498.24	0.58	859.0
Trefethen_20	OOM	0.23	0.0023	100.0
LFAT5	7.30	0.081	0.00045	180.0

Graph	SpWL-T (s)	GWL-T (s)	GWL-L (s)	GT (s)	ScaWL (s)	vs GT	vs GWL-T	vs GWL-L	vs SpWL-T
662_bus	16,957.48	327.62	352.16	4,074.55	7.23	563.6	45.3	48.7	2,345.4
494_bus	5,505.50	100.25	127.55	1,670.40	3.57	467.9	28.1	35.7	1,542.2
rbas480	3,506.95	70.12	116.71	1,616.02	3.03	533.3	23.1	38.5	1,157.4
celegansneural	602.26	9.85	21.76	158.49	0.58	273.3	17.0	37.5	1,038.4
Trefethen_20	0.058	0.0027	0.0057	0.037	0.0023	16.1	1.2	2.5	25.2
LFAT5	0.018	0.00053	0.0021	0.014	0.00045	31.1	1.2	4.7	40.0

Graph	SpWL-T (s)	GWL-T (s)	GWL-L (s)	GT (s)	ScaWL (s)	vs GWL-T	vs GWL-L	vs GT	vs SpWL-T
celegansneural	OOM	60,768.20	88,494.30	OOM	543.00	111.9	162.9	NA	NA
Trefethen_20	2.55	0.16	0.54	1.19	0.20	0.8	2.7	5.9	12.75
LFAT5	0.39	0.03	0.12	0.30	0.06	0.5	2.0	5.0	6.5

All run times are reported in seconds, and OOM indicates Out of Memory. Top table: Unpublished 2-WL, Middle table: Published 2-WL, Bottom table: Published 3-WL implementations.

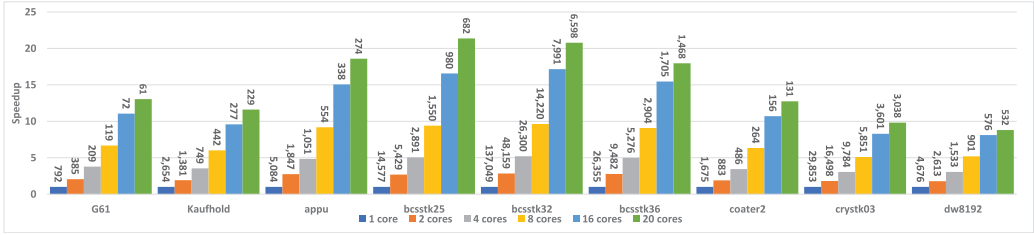


Fig. 7. 2-WL Single Node Part 1: Speedup with respect to a single thread. The number on top of each bar represents run time in seconds.

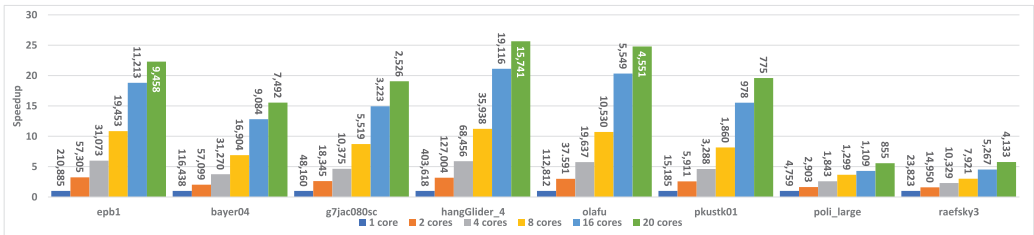


Fig. 8. 2-WL Single Node Part 2: Speedup with respect to a single thread. The number on top of each bar represents run time in seconds.

we mirror the edges explicitly, as the matrix market format only stores one of the edges. Next, we create the isomorphic version by reversing the node labels and outputting the relabeled version. The original graph and the relabeled graph pair are then subjected to the isomorphic test.

Figures 7 and 8 show the 2-WL single-node results. Figures 9 and 10 show the 3-WL single-node results.

Both the 2-WL and 3-WL single-node graphs omit data for graphs that did not complete because the node ran out of memory. In the multi-node graphs in the next section, the graphs that required more nodes to complete execution appear to the right of the red line, referred to as the node memory boundary.

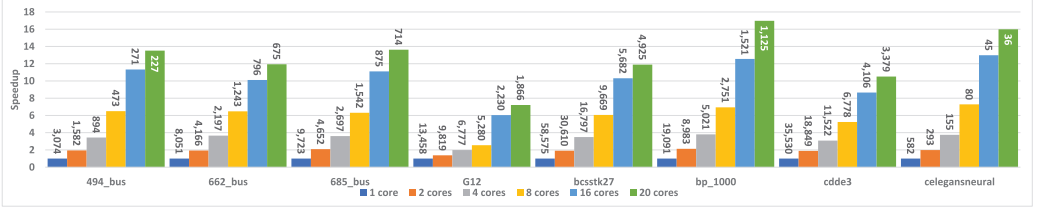


Fig. 9. 3-WL Single Node Part 1: Speedup with respect to a single thread. The number of top of each bar represents run time in seconds.

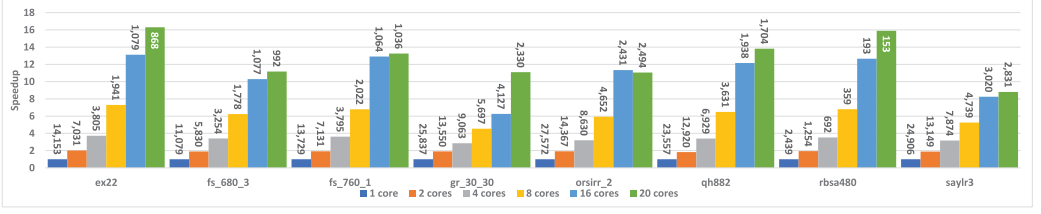


Fig. 10. 3-WL Single Node Part 2: Speedup with respect to a single thread. The number of top of each bar represents run time in seconds.

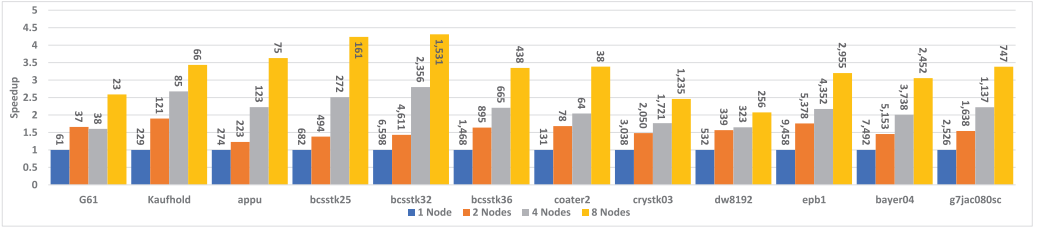


Fig. 11. 2-WL Distributed Node Part 1: Speedup with respect to a single node. The number on top of each bar represents run time in seconds.

The single-node performance of both the 2-WL and 3-WL algorithms shows substantial speedups and good scalability. The average speedup for 2-WL single-node tests is 2.38 for 2 cores, 4.26 for 4 cores, 7.64 for 8 cores, 13.20 for 16 cores, and 16.06 for 20 cores. The average speedup for 3-WL single-node tests is 1.91 for 2 cores, 3.36 for 4 cores, 6.04 for 8 cores, 10.63 for 16 cores, and 12.69 for 20 cores.

Overall, these results show that our single-node load-balancing strategies related to row/column and tuple coloring are effective at distributing the workload across the available cores within a single node.

6.4 Distributed Node Results

This section provides performance results and discussion for experiments with 2-WL and 3-WL simultaneously using distributed-node and single-node parallelism. The MPI library was used to perform the internode communication.

Figures 11 and 12 show results on the 2-WL distributed node runs, and Figures 13 and 14 show the corresponding results for the 3-WL case. The figures illustrate that many graphs significantly increase performance when increasing the node count. G12 and Bcsstk35 show a smaller degree of parallelism than the other graphs. These are cases where the initial partitioning of the graph

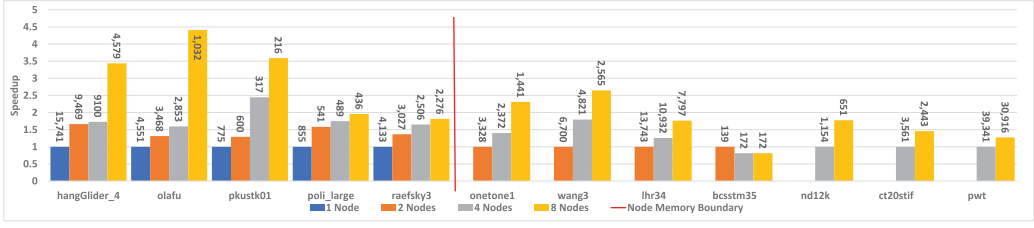


Fig. 12. 2-WL Distributed Node Part 2: Speedup with respect to first successful node count. The number on top of each bar represents run time in seconds. Right of red line: Node memory boundary designating graphs where multiple nodes were needed to avoid OOM test runs.

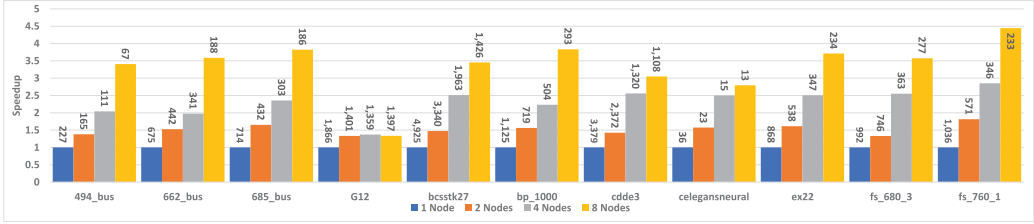


Fig. 13. 3-WL Distributed Node Part 1: Speedup with respect to a single node. The number on top of each bar represents run time in seconds.

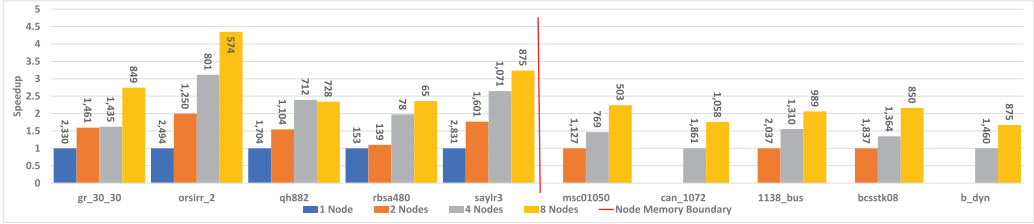


Fig. 14. 3-WL Distributed Node Part 2: Speedup with respect to first successful node count. The number on top of each bar represents run time in seconds. Right of red line: Node memory boundary designating graphs where multiple nodes were needed to avoid OOM test runs.

data is significantly skewed, coupled with a relatively small number of refinement iterations until convergence. This situation makes it hard for the load-balancing mechanisms of the algorithm to recover from the initial imbalance, reducing the overall parallelism achieved. The average speedup for 2-WL multi-node tests over one node is 1.53 for 2 nodes, 2.06 for 4 nodes, and 3.20 for 8 nodes. The average speedup for 3-WL multi-node tests is 1.54 for 2 nodes, 2.33 for 4 nodes, and 3.25 for 8 nodes. The graphs to the right of the node memory boundary are not included in the average speedup calculations because they lack a single-node reference time.

The multi-node experiments illustrate that our internode load-balancing heuristic and communication-avoiding design are effective, in the vast majority of cases, at scaling k -WL across multiple nodes. It efficiently handles large graphs, allowing them to be processed even when they exceed the capacity of fewer nodes (i.e., the graphs to the right of the node memory boundary).

7 Proof of Correctness

We provide in this section a formal proof to show that the classic k -WL algorithm and the proposed ScaWL algorithm are equivalent in terms of expressivity. The proof does so by showing that

Table 3. Definitions for the Proof of Correctness

Term	Definition
color	A label denoting a particular equivalence class
b	A formal bin created in every iteration of ScaWL
ib	An “invisible bin” in each iteration which corresponds to a formal bin in ScaWL, but is not formally partitioned
B_{classic}	The set of “invisible” bins that exist in the original k -WL algorithm at every iteration but are not formally partitioned as they are in the ScaWL algorithm
B_{scaWL}	The set of formally-created bins in the ScaWL algorithm that correspond to the “invisible” bins of the classic k -WL algorithm
$K_{\text{classic}_{n+1}}$	The set of keys that exist in the original k -WL algorithm at iteration $n + 1$
$K_{\text{scaWL}_{n+1}}$	The set of keys that exist in the ScaWL algorithm at iteration $n + 1$
EC_{scaWL_n}	The set of all equivalence classes generated at iteration n by ScaWL
EC_{classic_n}	The set of all equivalence classes generated at iteration n by classical k -WL
$EC_{\text{scaWL}_{n+1}}$	The set of all equivalence classes generated at iteration $n + 1$ by ScaWL
$EC_{\text{classic}_{n+1}}$	The set of all equivalence classes generated at iteration $n + 1$ by classical k -WL
EC_{scaWL}	A particular equivalence class in the new algorithm
EC_{classic}	A particular equivalence class in the classic k -WL algorithm
ec	A set containing k -tuple members of a particular equivalence class
HASH	The k -WL hash function that produces the new equivalence classes of the next iteration
HASH_BIN	Performing the k -WL HASH function on each k -tuple of a bin
F	A bijection between an equivalence class of the classical k -WL and an equivalence class in ScaWL such that $EC_{\text{scaWL}} = F(EC_{\text{classic}})$. The function preserves k -tuple membership in a particular equivalence class, but the members may receive a new color label.
K	A bijection between the keys in the classic k -WL algorithm and ScaWL, denoting a one-to-one correspondence between the key sets in both algorithms
NC_i	A bijection between the colors of neighborhood i in the classic k -WL algorithm and ScaWL, denoting a one-to-one correspondence between the neighborhood i 's colors in both algorithms
UM_i	The number of unique k -tuple-color multisets for neighborhood i at iteration n

both the equivalence classes and k -tuple membership in those classes remain identical in both algorithms, ensuring their expressivity is the same at each iteration. The analysis focuses on the k -tuple neighborhood $i = 2$ (row dimension), but this choice is arbitrary, and the proof can be generalized to any i -th neighborhood.

The proof employs induction on the iterations of the k -WL algorithm. The definitions needed throughout the proof are collected in Table 3.

7.1 Equivalence in Expressivity between ScaWL and Classic k -WL

This inductive proof has three primary goals: establishing key equivalency between ScaWL and k -WL, preserving key and color integrity in both single-node and distributed settings, and demonstrating that the ScaWL and k -WL have matching equivalence classes with identical k -tuple membership at each iteration. The goals are clearly demarcated as the proof progresses.

7.1.1 Base Case. Since the initial colors are identical in the first iteration, both the scaWL algorithm and the classical k -WL have the same color labels— hence, the same equivalence classes and k -tuple membership in each equivalent class.

7.1.2 Inductive Hypothesis. Assume that at iteration n of both the classic k -WL and the scaWL algorithm, there exists the same number of equivalence classes and the same k -tuple members within those equivalence classes. The following invariants hold (see definitions in Table 3):

$$|\mathbf{EC}_{\text{scaWL}_n}| = |\mathbf{EC}_{\text{classic}_n}| \quad (4)$$

$$\forall ec_2 \in \mathbf{EC}_{\text{classic}_n}, \exists ! ec_1 \in \mathbf{EC}_{\text{scaWL}_n} : ec_1 = F(ec_2) \quad (5)$$

7.1.3 Inductive Step. Our goal is to show that the invariant holds at iteration $n + 1$.

One difference between the classic k -WL algorithm and ScaWL lies in the neighborhood color labeling ($c_{v,i}^{(l)}$ and $d_{u,i}^{(l)}$ from Algorithm 1, lines 6 and 8. The classic k -WL algorithm colors neighborhood multisets that match the same color. However, upon close inspection of the key construction step in Algorithm 1, lines 7 and 9, it is clear that the i -th neighborhood of all the k -tuples of G and H needs to be colored consistently, but color consistency across neighborhoods is unnecessary. In our 2-WL examples, this translates into all the rows of G and H being colored consistently and all the columns of G and H being colored consistently, but the rows and columns are not required to be consistent with each other. Recall that rows correspond to the multisets generated by neighborhood $i = 2$ in Line 6 and Line 8 of Algorithm 1.

The ScaWL coloring technique will produce an outcome that is equivalent to classic k -WL, as demonstrated by the following example. Consider two neighborhoods, i and $i + 1$, both containing the same multiset M . Classic k -WL will color M the same color C_{classic} in both neighborhoods. ScaWL assigns multiset M potentially different colors C_{ScaWL} and C'_{ScaWL} in neighborhood i and $i + 1$, respectively. So far, we have $\text{HASH} < \dots C_{\text{classic}}, C_{\text{classic}} \dots >$ in classic k -WL and $\text{HASH} < \dots C_{\text{ScaWL}}, C'_{\text{ScaWL}} \dots >$ in ScaWL.

Now note that $C_{\text{classic}}, C_{\text{classic}}$ and $C_{\text{ScaWL}}, C'_{\text{ScaWL}}$ denote the exact same **relative structural similarities or differences** between G and H and, therefore, encode the same structural information within the hash key. The equivalence to the classic k -WL algorithm can be demonstrated by showing how the ScaWL HASH function relates to the classic k -WL HASH function.

Recalling the HASH functions from lines 7 and 9 of Algorithm 1, and letting NC_i be a bijection between neighborhood i 's color in classic k -WL and ScaWL, we have:

LEMMA 1. NC_i is injective because every color in neighborhood i of the classic k -WL algorithm has a one-to-one mapping to a color in neighborhood i of ScaWL. NC_i is surjective because every color in neighborhood i of ScaWL has a one-to-one mapping to a color in neighborhood i of classic k -WL. Therefore, NC_i is bijective.

PROOF. Let the number of unique k -tuple-equivalence-class multisets for neighborhood i at iteration n be UM_i . Furthermore, the ScaWL framework and classic k -WL both injectively map each unique multiset to a color. Clearly, the ScaWL framework explicitly performs this mapping from each unique multiset to a color. The classic k -WL algorithm performs the mapping implicitly by key construction (Figure 4). In doing so, the respective color outputs of both classic k -WL and scaWL will have a one-to-one and onto mapping, proving NC_i 's existence. $\square$

It follows from Lemma 1 that:

THEOREM 1.

$$\begin{aligned} \text{HASH} \langle c_v^{l-1}, c_{v,1}^l, c_{v,2}^l, \dots, c_{v,k}^l : \forall v \in V \rangle = \\ \text{HASH} \langle c_v^{l-1}, NC_1(c_{v,1}^l_{\text{ScaWL}}), NC_2(c_{v,2}^l_{\text{ScaWL}}), \dots, NC_k(c_{v,k}^l_{\text{ScaWL}}) : \forall v \in V \rangle \end{aligned} \quad (6)$$

$$\begin{aligned} \text{HASH} \langle d_u^{l-1}, d_{u,1}^l, d_{u,2}^l, \dots, d_{u,k}^l : \forall u \in U \rangle = \\ \text{HASH} \langle d_u^{l-1}, NC_1(d_{u,1}^l_{\text{ScaWL}}), NC_2(d_{u,2}^l_{\text{ScaWL}}), \dots, NC_k(d_{u,k}^l_{\text{ScaWL}}) : \forall u \in U \rangle \end{aligned} \quad (7)$$

PROOF. Because each NC_i is a bijective function mapping neighborhood i 's color between classic k -WL and ScaWL, the new key formulation guarantees equivalent hashing outcomes between the two algorithms. In the 2-WL examples from this paper, NC_1 and NC_2 are established by column coloring and row coloring, respectively.

It is important to note that each NC_i is applied consistently to neighborhood i of every k -tuple across both graphs G and H . This consistency is crucial for maintaining the encoding of the same structural information as classic k -WL. $\square$

COROLLARY 1. *For the single-node scenario, since Theorem 1 shows that the hashing of ScaWL is equivalent to classic k -WL, it follows that every key in classic k -WL has a one-to-one mapping to a key in ScaWL; therefore, K is injective. K is also surjective because every key in ScaWL has a one-to-one mapping to a key in classic k -WL. Therefore, K is bijective.*

COROLLARY 2. *For the single-node scenario, since keys in turn have a one-to-one mapping to equivalence classes, F is both injective and surjective, mirroring key bijectivity exactly. Hence, F is also bijective.*

Established k -WL Hash Key Equivalencies

Given that the inputs to the classic k -WL and ScaWL algorithm encode equivalent structural information, their outputs will produce the same equivalence classes.

The next step of the proof is to show that the bijectivity of both K and F hold when the algorithm distributes data across nodes by ensuring no two nodes generate the same key.

Notice that the key construction contains the row color ($c_{v,2ScaWL}^l$ and $d_{u,2ScaWL}^l$ for G and H , respectively) within it and that the ScaWL algorithm places the rows in a bin via a row hashing algorithm. It follows that:

LEMMA 2. *Rows within a bin cannot generate k -tuple hash keys that collide with k -tuple hash keys of rows in other bins.*

PROOF. Assume that row r_1 in bin b_1 contains k -tuple t_1 and row r_2 in bin b_2 contains k -tuple t_2 and both k -tuples' keys match. This implies that rows containing k -tuple t_1 and t_2 have the same color. However, since row color is used in the k -WL key construction and the non-injective hash used in binning means the rows cannot have the same color, the hash keys for k -tuple t_1 and t_2 cannot match. $\square$

LEMMA 3. *From Lemma 2, rows within a bin cannot generate new k -tuple colors that collide with the k -tuple colors of rows in other bins, since a k -tuple's color maps directly to its key.*

Consequently,

LEMMA 4. *The color space of any group of row bins is completely isolated from any other group of row bins.*

Lemmas 2 through 4 provide the foundation for building and maintaining an isolated color space, which is needed for distributed computation. From these lemmas, Theorem 2 follows:

THEOREM 2. *Transfers between compute nodes at the row-bin granularity are safe because there is a guarantee of no key collisions with any k -tuple outside the row bin.*

LEMMA 5. *It follows from Lemma 4 that, since each compute node contains a group of row bins, its color space is isolated from the color spaces of other nodes.*

PROOF. Lemma 5 allows for a non-colliding global color space solution implemented across all compute nodes. A broadcast will occur among all compute nodes regarding each compute node's unique key counts, and those counts will be used by each node to calculate their color offset for their keys. This ensures that color assignments for hashed keys do not collide across nodes. $\square$

LEMMA 6. *Following from Lemma 5, it is impossible for colors to be lost from inter-node data transfer because the same key and color cannot exist on different nodes, ensuring correct color count aggregation.*

An essential aspect of the correctness of ScaWL is that the number of new colors generated at each iteration matches the classic k -WL algorithm. Lemma 6 ensures that this condition is satisfied.

LEMMA 7. *Following from Lemma 5, it is impossible for the k -tuple colors of the previous iteration to collide across compute nodes.*

Lemma 7 is crucial because the color from the previous iteration is used in the construction of the key in Theorem 1. From Theorem 1 and Lemmas 5–7, Theorem 3 below follows:

THEOREM 3. $|K_{classic_{n+1}}| = |K_{scaWL_{n+1}}|$ and $K_{scaWL_{n+1}} = \{K(k) : k \in K_{classic_{n+1}}\}$

Since each unique key is mapped to its own unique color, and hence, an equivalence class, it follows from Theorem 3 that:

COROLLARY 3. $|EC_{scaWL_{n+1}}| = |EC_{classic_{n+1}}|$

k -WL Hash Key Integrity Guarantees

The integrity of the hash keys of the k -WL algorithm is maintained by constructing the hash keys so that no matching keys—and their corresponding matching colors—are ever created on different compute nodes. This ensures key-equivalency between classic k -WL and scaWL is preserved in a distributed context.

The next task is to show that for each equivalence class in the k -WL algorithm that has a bijective mapping to an equivalence class in ScaWL, the two equivalence classes have the same k -tuple membership at each iteration. Consider a row bin called b in ScaWL and its corresponding rows in the classic k -WL, which will be called an “invisible bin” or ib . It is clear that:

LEMMA 8. *Every row bin b in ScaWL will have a corresponding “invisible bin” ib in standard k -WL at iteration n .*

Since ib and b contain the same rows, the only potential difference between b and ib is in their color assignments because the formal row bin b may reside on another compute node. However, because both b and ib contain the same rows and k -tuples, those k -tuples will be partitioned into the same equivalence classes, and each equivalence class within the bin should have the same k -tuple membership.

LEMMA 9. *From Theorem 1, Lemmas 2 and 3, new equivalence classes generated by performing HASH_BIN on a bin will be complete. No new k -tuple members within the newly generated equivalence classes will come from outside the bin.*

LEMMA 10. *From Lemma 9, invoking the HASH_BIN function on a bin produces new complete equivalence classes whose membership is completely comprised of a partition of the bin's k -tuples.*

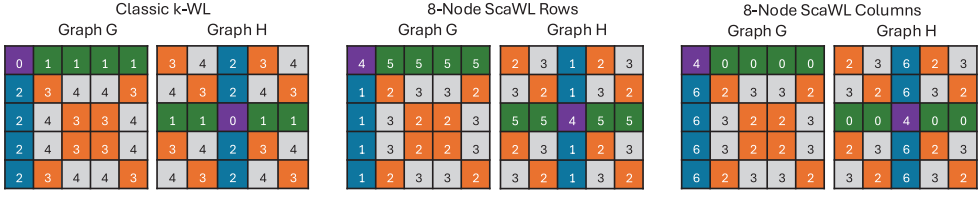


Fig. 15. Left: Classic k -WL showing the equivalence class partition of the two isomorphic graphs G and H . Middle and Right: The ScaWL algorithm shows the equivalence class partition of G and H for the 8-node configuration.

Lemma 9 ensures that newly generated equivalence classes are not split across compute nodes at any iteration. If this were to happen, k -tuple members of the same equivalence class could receive different color assignments, resulting in an error.

LEMMA 11. *At iteration n , the bins b and ib must contain the same equivalence classes and k -tuple membership in each equivalence class.*

The bijective mapping between the equivalence classes is given by F . Following from Lemma 11:

THEOREM 4. $b = \{F(ec) | ec \in ib\}$

LEMMA 12. *From Theorem 1, Lemma 10, and Lemma 11, if the two bins b and ib have the same equivalence classes and k -tuple membership within each equivalence class, then performing HASH_BIN on bins b and ib will produce the same number of new equivalence classes. Each new equivalence class will have the same k -tuple membership.*

Lemmas 10 and 12 ensure that both classic k -WL and ScaWL generate the same equivalence classes and corresponding k -tuple memberships at each iteration. Therefore:

LEMMA 13. *Combining Theorem 1, Theorem 4, and Lemma 12:*

$$\forall ib \in B_{classic_n}, \exists! b \in B_{ScaWL_n} : HASH_BIN(b) = \{F(ec) | ec \in HASH_BIN(ib)\}$$

LEMMA 14.

$$EC_{ScaWL_{n+1}} = \{HASH_BIN(b) : b \in B_{ScaWL_n}\}$$

$$EC_{classic_{n+1}} = \{HASH_BIN(ib) : ib \in B_{classic_n}\}$$

From Lemmas 12–14, the number of new equivalence classes and their corresponding k -tuple membership should be the same for b and ib at iteration $n + 1$. It follows that:

THEOREM 5. $\forall ec_2 \in EC_{classic_{n+1}}, \exists! ec_1 \in EC_{ScaWL_{n+1}} : ec_1 = F(ec_2)$

Hence, by Corollary 3 and Theorem 5 the following invariants hold at iteration $n + 1$:

$$|EC_{ScaWL_{n+1}}| = |EC_{classic_{n+1}}| \quad (8)$$

$$\forall ec_2 \in EC_{classic_{n+1}}, \exists! ec_1 \in EC_{ScaWL_{n+1}} : ec_1 = F(ec_2) \quad (9)$$

Since the same equivalence classes and their k -tuple membership hold at every iteration, the ScaWL algorithm is equivalent in expressivity to classic k -WL.

Figure 15 shows the isomorphic graphs G and H from Figure 1. It shows the equivalence between G and H for both the rows and columns in ScaWL using the 8-node configuration. You can see that the equivalence classes, shown in the different cell shadings, are the same between k -WL and ScaWL. The color assignments are represented by the integers within the cells.

Equivalence Classes Maintained

The combination of key equivalency between k -WL and ScaWL and the maintenance of key and color integrity in a distributed setting implies that ScaWL is guaranteed to create the same number of equivalence classes, each with the same k -tuple membership, at each iteration of k -WL.

8 Conclusion

In this work, we uncover and exploit key insights from the k -WL algorithm—an important kernel used in a range of applications, including Graph Neural Networks—to deliver an implementation with an order of magnitude lower data footprint and an order of magnitude lower communication volume than existing implementations. Our ScaWL approach provides high performance and high internode and intranode scalability, while maintaining the same accuracy guarantee as the conventional k -WL algorithm. To the best of our knowledge, this is the first work that focuses on performance aspects of the k -WL algorithm and the first work that delivers a distributed implementation.

References

- [1] Eren Akbiyik, Florian Grötschla, Béni Egressy, and Roger Wattenhofer. 2023. Graphtester: Exploring theoretical boundaries of GNNs on graph datasets. In *Data-centric Machine Learning Research (DMLR) Workshop at ICML 2023, Honolulu, Hawaii*.
- [2] G. Ballard, E. Carson, J. Demmel, M. Hoemmen, N. Knight, and O. Schwartz. 2014. Communication lower bounds and optimal algorithms for numerical linear algebra. *Acta Numerica* 23 (2014), 1–155. <https://doi.org/10.1017/S0962492914000038>
- [3] Cristian Bodnar, Fabrizio Frasca, Yu Guang Wang, Nina Otter, Guido Montúfar, Pietro Liò, and Michael M. Bronstein. 2021. Weisfeiler and Lehman go topological: Message passing simplicial networks. *CoRR* abs/2103.03212 (2021). arXiv:2103.03212 <https://arxiv.org/abs/2103.03212>
- [4] Gian Maria Campedelli, Janet Layne, Jack Herzoff, and Edoardo Serra. 2021. The geometrical shapes of violence: Predicting and explaining terrorist operations through graph embeddings. 10 (Mar. 2021). <https://doi.org/10.1093/comnet/cnac008>
- [5] Clemens Damke, Vitalik Melnikov, and Eyke Hüllermeier. 2020. A novel higher-order Weisfeiler-Lehman graph convolution. *CoRR* abs/2007.00346 (2020). arXiv:2007.00346 <https://arxiv.org/abs/2007.00346>
- [6] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (Dec. 2011), 25 pages. <https://doi.org/10.1145/2049662.2049663>
- [7] James Demmel, Mark Hoemmen, Marghoob Mohiyuddin, and Katherine Yelick. 2008. Avoiding communication in sparse matrix computations. In *2008 IEEE International Symposium on Parallel and Distributed Processing*. 1–12. <https://doi.org/10.1109/IPDPS.2008.4536305>
- [8] Martin Fürer. 2001. Weisfeiler-Lehman refinement requires at least a linear number of iterations. In *Automata, Languages and Programming*, Fernando Orejas, Paul G. Spirakis, and Jan van Leeuwen (Eds.). Springer Berlin, Berlin, 322–333.
- [9] Martin Grohe and Pascal Schweitzer. 2020. The graph isomorphism problem. *Commun. ACM* 63, 11 (Oct. 2020), 128–134. <https://doi.org/10.1145/3372123>
- [10] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation learning on graphs: Methods and applications. <http://arxiv.org/abs/1709.05584>
- [11] Hong Jia-Wei and H. T. Kung. 1981. I/O complexity: The red-blue pebble game. In *Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing* (Milwaukee, WI, USA) (STOC '81). Association for Computing Machinery, New York, NY, USA, 326–333. <https://doi.org/10.1145/800076.802486>
- [12] Mikel Joaristi and Edoardo Serra. 2021. SIR-GN: A fast structural iterative representation learning approach for graph nodes. *ACM Trans. Knowl. Discov. Data* 15, 6, Article 100 (May 2021), 39 pages. <https://doi.org/10.1145/3450315>
- [13] Mikel Joaristi and Edoardo Serra. 2022. Structural iterative lexicographic autoencoded node representation. *Data Mining and Knowledge Discovery* 37 (11 2022). <https://doi.org/10.1007/s10618-022-00880-x>
- [14] Tiago Toledo Junior. 2022. Paper Implementations. https://github.com/TNanukem/paper_implementations
- [15] Moritz Lichter, Ilia Ponomarenko, and Pascal Schweitzer. 2019. Walk refinement, walk logic, and the iteration number of the Weisfeiler-Leman algorithm. *CoRR* abs/1905.03008 (2019). arXiv:1905.03008 <http://arxiv.org/abs/1905.03008>

- [16] Haggai Maron, Heli Ben-Hamu, Hadar Serviansky, and Yaron Lipman. 2019. Provably powerful graph networks. *CoRR* abs/1905.11136 (2019). arXiv:1905.11136 <http://arxiv.org/abs/1905.11136>
- [17] Christopher Morris, Matthias Fey, and Nils M. Kriege. 2021. The power of the Weisfeiler-Leman algorithm for machine learning with graphs. *CoRR* abs/2105.05911 (2021). arXiv:2105.05911 <https://arxiv.org/abs/2105.05911>
- [18] Christopher Morris, Kristian Kersting, and Petra Mutzel. 2017. Glocalized Weisfeiler-Lehman graph kernels: Global-local feature maps of graphs. In *IEEE International Conference on Data Mining (ICDM)*, 2017.
- [19] Christopher Morris, Gaurav Rattan, and Petra Mutzel. 2020. Weisfeiler and Leman go sparse: Towards scalable higher-order graph embeddings. arXiv:1904.01543 [cs.DS] <https://arxiv.org/abs/1904.01543>
- [20] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. 2018. Weisfeiler and Leman go neural: Higher-order graph neural networks. *CoRR* abs/1810.02244 (2018). arXiv:1810.02244 <http://arxiv.org/abs/1810.02244>
- [21] Ryoma Sato. 2020. A survey on the expressive power of graph neural networks. *CoRR* abs/2003.04078 (2020). arXiv:2003.04078 <https://arxiv.org/abs/2003.04078>
- [22] Ryoma Sato, Makoto Yamada, and Hisashi Kashima. [n. d.]. *Random Features Strengthen Graph Neural Networks*. 333–341. <https://doi.org/10.1137/1.9781611976700.38> arXiv:<https://epubs.siam.org/doi/pdf/10.1137/1.9781611976700.38>
- [23] Edoardo Serra. 2024. K-FWL. <https://github.com/edoardoserra/K-F-WL->
- [24] Edoardo Serra, Mikel Joaristi, and Alfredo Cuzzocrea. 2020. Large-scale sparse structural node representation. In *2020 IEEE International Conference on Big Data (Big Data'20)*. 5247–5253. <https://doi.org/10.1109/BigData50022.2020.9377854>
- [25] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. 2011. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research* 12, 77 (2011), 2539–2561. <http://jmlr.org/papers/v12/shervashidze11a.html>
- [26] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. 2011. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research* 12, 77 (2011), 2539–2561. <http://jmlr.org/papers/v12/shervashidze11a.html>
- [27] B. Weisfeiler and A. A. Lehman. 1968. The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Tekhnicheskaya Informatsia* 2, 9 (1968), 12–16.
- [28] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=ryGs6iA5Km>
- [29] Zuoyu Yan, Tengfei Ma, Liangcai Gao, Zhi Tang, Chao Chen, and Yusu Wang. 2023. Cycle invariant positional encoding for graph representation learning. arXiv:2311.14333 [cs.LG] <https://arxiv.org/abs/2311.14333>
- [30] Bingxue Zhang, Changjun Fan, Shixuan Liu, Kuihua Huang, Xiang Zhao, Jin-Yu Huang, and Zhong Liu. 2023. The expressive power of graph neural networks: A survey. *ArXiv* abs/2308.08235 (2023). <https://api.semanticscholar.org/CorpusID:260925813>
- [31] Bohang Zhang, Jingchu Gai, Yiheng Du, Qiwei Ye, Di He, and Liwei Wang. 2024. Beyond Weisfeiler-Lehman: A quantitative framework for GNN expressiveness. arXiv:2401.08514 [cs.LG] <https://arxiv.org/abs/2401.08514>

Received 25 May 2024; revised 20 November 2024; accepted 29 December 2024